

LAB WORKBOOK

Assignment 2: Synchronization & Explicit Waits

Tier 1: Core Fundamentals & Locators

1. Module Name and Experiment Title

Module: Tier 1 – Core Fundamentals & Locators

Experiment: Synchronization & Explicit Waits

2. Problem Statement

Automate a web page containing dynamic content where a Start button triggers a delayed response. The program must wait for the dynamic content to appear using Selenium WebDriverWait and expected_conditions. The use of time.sleep() is not permitted.

3. Objective

To understand synchronization in Selenium and learn how to use explicit waits to reliably interact with elements that appear or become visible after a delay.

4. Tools, Software and Concepts Used

Category	Details
Programming Language	Python 3
Automation Tool	Selenium WebDriver
Browser	Google Chrome
IDE/Editor	VS Code / PyCharm
Website	The Internet – Dynamic Loading Example
Concepts	WebDriverWait, expected_conditions, element_to_be_clickable, visibility_of_element_located, synchronization

5. Source Code / Implementation

The following solution deliberately avoids time.sleep() and uses explicit waits to synchronize the automation with the dynamic page.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Launch Chrome
driver = webdriver.Chrome()
wait = WebDriverWait(driver, 15)
```

```
try:
    # Open the dynamic loading page
    driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")

    # Wait until the Start button is clickable, then click it
    start_button = wait.until(
        EC.element_to_be_clickable((By.CSS_SELECTOR, "#start button"))
    )
    start_button.click()

    # Wait until the dynamically loaded text becomes visible
    finish_text = wait.until(
        EC.visibility_of_element_located((By.ID, "finish"))
    )

    # Validate the displayed text
    assert finish_text.text == "Hello World!", (
        "Dynamic content did not appear as expected."
    )

    print("Dynamic content loaded successfully.")
    print("Validation passed: Hello World! is visible.")

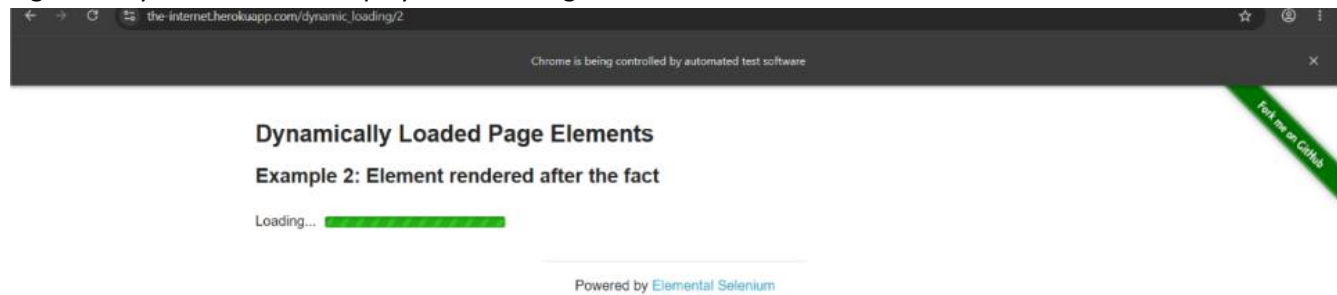
finally:
    driver.quit()
```

6. Implementation Steps

1. Open the dynamic loading example page in Chrome.
2. Create a WebDriverWait object with a maximum timeout of 15 seconds.
3. Wait until the Start button is clickable using `expected_conditions.element_to_be_clickable`.
4. Click the Start button to trigger the dynamic content.
5. Wait until the element containing the result is visible using `expected_conditions.visibility_of_element_located`.
6. Verify that the displayed text is Hello World!.
7. Print the validation result and close the browser.

7. Output

Figure 1: Dynamic content displayed after using WebDriverWait.



Screenshot 1: Loading bar visible on screen

8. Result

The dynamic content was successfully detected and validated using WebDriverWait and expected_conditions without using time.sleep().

9. Observation

Explicit waits allow Selenium to wait only until a required condition is satisfied, making automation more reliable than using fixed delays. The test can continue as soon as the required element becomes ready.

10. Conclusion

This experiment demonstrated Selenium synchronization using explicit waits. WebDriverWait combined with expected_conditions provides a reliable way to handle dynamic web elements and reduces failures caused by timing differences.